Domænemodel v1

Udlejningssystem

Status: Første godkendte udkast
Backend: C# / ASP.NET Core Web API
Database: Microsoft SQL Server med EF Core
Frontend: Angular / TypeScript
Arkitektur: Modulær monolit

---

1. Formål og afgrænsning

Domænemodellen beskriver systemets vigtigste data, relationer, ansvarsområder og forretningsregler.

Den er en fælles reference for implementeringen. Den er ikke en komplet specifikation af alle C#-klasser, properties, interfaces eller API-endpoints. Disse detaljer fastlægges løbende, når de relevante funktioner implementeres.

Omfattet af V1

- Brugere, roller og autentifikation.
- Afdelinger og boliger.
- Billeder af boliger.
- Reservationer, bookinger og betalinger.
- Refunderinger.
- Kundehenvendelser og officielle fejlmeldinger.
- Notifikationer og e-mails.
- Administration og bookingoversigt.

IoT, RFID og mikroserviceopdeling er planlagt til senere versioner.

2. Entiteter og ansvar

2.1 ApplicationUser

Repræsenterer alle brugere af systemet gennem ASP.NET Core Identity.

Ansvar

- Autentifikation og brugeridentitet.
- Kundeprofil med navn, e-mail og telefonnummer.
- Roller og adgangskontrol.

Roller

- `Customer`
- `Administrator`

Vi bruger én fælles brugermodel frem for separate kundetabeller og administratortabeller. Administratoradgang styres gennem roller og autorisation.

Kunder skal bekræfte deres e-mail, før de kan anvende beskyttede funktioner som booking. Ubekræftede konti slettes efter 24 timer.

2.2 Department

Repræsenterer en afdeling, som administrerer boliger.

Ansvar

- Gruppere boliger efter afdeling.
- Give administratorer overblik over boliger på tværs af afdelinger.

En afdeling kan have flere boliger. En bolig tilhører én afdeling.

2.3 Home

Repræsenterer en bolig, som kunder kan leje.

Ansvar

- Boligens navn og beskrivelse.
- Aktuel dagspris i DKK.
- Kapacitet, antal værelser, areal og øvrige relevante egenskaber.
- Publicering og mulighed for at booke.
- Relationer til billeder, bookinger og afdeling.

Vigtige felter

- `IsPublished`: Om boligen vises for kunder.
- `IsBookable`: Om der må oprettes nye bookinger på boligen.

Felterne har forskellige formål. En offentliggjort bolig kan være midlertidigt utilgængelig for nye bookinger.

`IsBookable` erstatter ikke kontrollen af eksisterende bookinger. Backend skal altid kontrollere, om de ønskede datoer er ledige.

V1 har ikke en separat entitet til planlagte utilgængelighedsperioder. Det kan tilføjes senere, hvis behovet opstår.

2.4 HomeImage

Repræsenterer et billede tilknyttet en bolig.

Ansvar

- Reference til billedfilen.
- Relation til boligen.
- Rækkefølge for visning af billeder.

I V1 gemmes billeder lokalt, mens databasen gemmer en relativ filreference eller lagringsnøgle. Vi undgår absolutte filstier, som kun fungerer på én maskine.

Senere kan billeder flyttes til cloud-lagring eller en ekstern billedtjeneste uden nødvendigvis at ændre domænemodellen.

2.5 Booking

Repræsenterer hele bookingforløbet, fra en midlertidig reservation til et afsluttet eller annulleret ophold.

Vi bruger én `Booking`-entitet. Midlertidige reservationer er bookinger med status `PendingPayment` og en udløbstid.

Ansvar

- Kunde og bolig.
- Ankomst- og afrejsedato.
- Bookingstatus.
- Aftalt dagspris og samlet beløb.
- Reservationens udløbstid.
- Oplysninger om annullering.
- Oprettelsestidspunkt.

Vigtige data

- `ArrivalDate`
- `DepartureDate`
- `Status`
- `AgreedDailyPrice`
- `TotalAmount`
- `ExpiresAt`, når reservationen afventer betaling.
- `CreatedAt`
- `CancelledAt`, hvis bookingen annulleres.
- `CancelledByUserId`, hvis bookingen annulleres.
- `CancellationReason`, hvis en årsag registreres.

Datoer gemmes som kalenderdatoer (`DateOnly` i C#). Beløb gemmes som `decimal`, og V1 bruger DKK.

Bookingens pris gemmes som et øjebliksbillede. Prisændringer på boligen påvirker ikke eksisterende bookinger.

2.6 Payment

Repræsenterer et enkelt betalingsforsøg knyttet til en booking.

En booking kan have flere betalingsforsøg. Mislykkede og vellykkede forsøg bevares i historikken.

Ansvar

- Betalingsbeløb.
- Betalingsstatus.
- Oprettelsestidspunkt.
- Identifikation af betalingstransaktioner, når relevant.

Bookingstatus og betalingsstatus er uafhængige. En udløbet booking kan eksempelvis have en betaling, der er gennemført for sent.

V1 bruger en betalingssimulator. Der gemmes ingen rigtige kortoplysninger.

2.7 Refund

Repræsenterer en refundering knyttet til en konkret betaling.

Ansvar

- Refunderet beløb.
- Refunderingens status.
- Oprettelsestidspunkt.
- Eventuel ekstern reference til refunderingen.

En betaling kan have flere refunderinger, hvis der er behov for det. Systemet skal holde styr på samlet refunderet beløb og undgå utilsigtede dobbeltrefunderinger.

En refundering betragtes først som gennemført, når resultatet er bekræftet.

2.8 Inquiry

Repræsenterer en henvendelse fra en kunde.

Ansvar

- Kundens oprindelige besked.
- Afsender.
- Eventuel relation til en booking.
- Oprettelsestidspunkt.

- Om administratoren har læst henvendelsen.

`BookingId` er valgfrit. Generelle spørgsmål kan oprettes uden en bookingrelation.

V1 bruger et simpelt `IsRead`-flag frem for en omfattende statusmodel.

En henvendelse slettes ikke automatisk, fordi administratoren har læst den eller oprettet en fejlmelding ud fra den.

2.9 FaultReport

Repræsenterer en officiel fejlmelding, som administratoren opretter, når et problem skal registreres og håndteres.

Ansvar

- Beskrivelse af problemet.
- Relation til den relevante bolig og eventuelt en booking.
- Eventuel relation til den oprindelige kundehenvendelse.
- Status og oprettelsestidspunkt.
- Oplysninger om håndtering.

Statusser

- `Open`
- `InProgress`
- `Resolved`

En kundehenvendelse bliver ikke automatisk til en officiel fejlmelding. Administratoren vurderer henvendelsen og opretter en `FaultReport`, hvis det er nødvendigt.

IoT-advarsler skal ligeledes kunne registreres separat uden automatisk at blive til officielle fejlmeldinger.

2.10 Notifications

Notifikationer er et ansvarligt modul, men behøver ikke nødvendigvis en selvstændig domæneentitet for alle typer meddelelser.

Modulet håndterer, hvordan notifikationer sendes, eksempelvis via e-mail. Andre moduler bestemmer, hvornår en notifikation skal sendes.

Eksempler:

- E-mailbekræftelse.
- Bookingbekræftelse.
- Påmindelse om ankomst eller afrejse.
- Besked om betalingsresultat.
- Notifikation til administrator om nye kundehenvendelser.

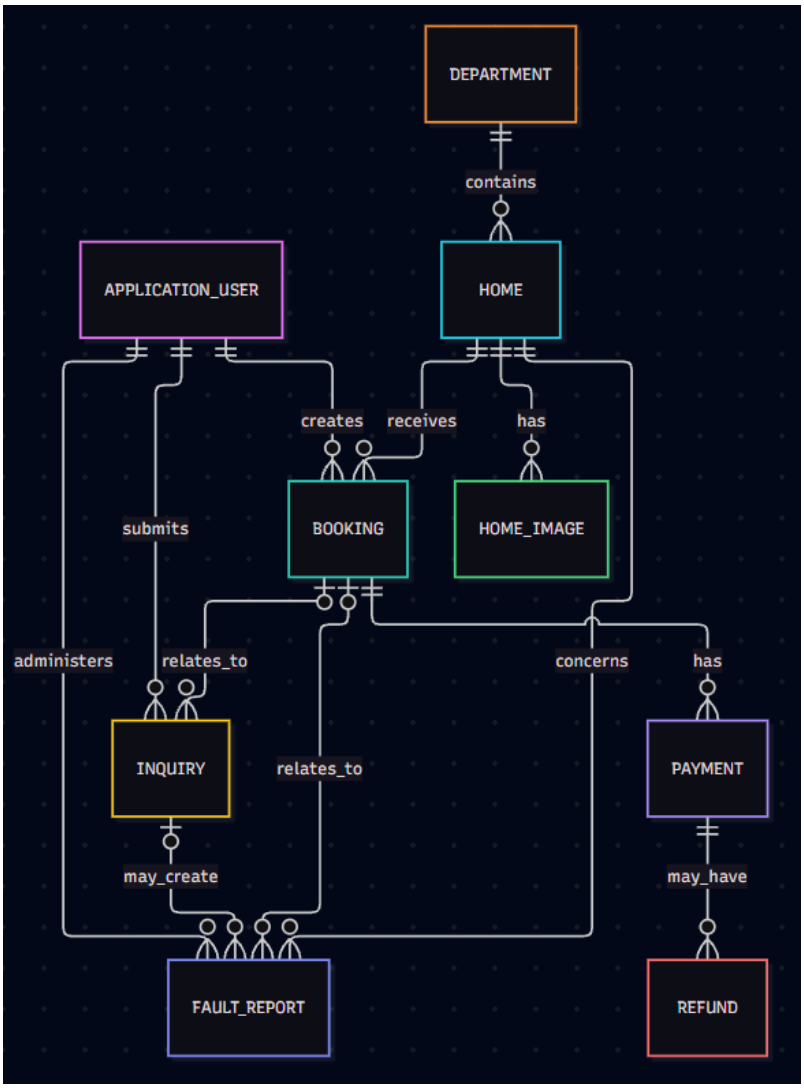
IoT-advarsler skal i en senere version kunne gemmes permanent og markeres som håndteret uden at slette den oprindelige log.

---

3. Relationer mellem entiteterne

Mermaid.live kode:

```
erDiagram
    APPLICATION_USER ||--o{ BOOKING : creates
    DEPARTMENT ||--o{ HOME : contains
    HOME ||--o{ HOME_IMAGE : has
    HOME ||--o{ BOOKING : receives
    BOOKING ||--o{ PAYMENT : has
    PAYMENT ||--o{ REFUND : may_have
    APPLICATION_USER ||--o{ INQUIRY : submits
    BOOKING o|--o{ INQUIRY : relates_to
    INQUIRY o|--o{ FAULT_REPORT : may_create
    BOOKING o|--o{ FAULT_REPORT : relates_to
    HOME ||--o{ FAULT_REPORT : concerns
    APPLICATION_USER ||--o{ FAULT_REPORT : administers
```



Diagrammet viser de overordnede relationer, ikke alle databasebegrænsninger. Valgfri relationer og detaljerede kardinaliteter fastlægges i implementeringen.

Relationelle principper

- En bruger kan have mange bookinger.

- En bolig tilhører én afdeling og kan have mange billeder.
- En bolig kan have mange bookinger over tid.
- En booking kan have flere betalingsforsøg.
- En betaling kan have flere refunderinger.
- En kundehenvendelse kan være knyttet til en booking, men behøver ikke være det.
- En officiel fejlmelding kan opstå fra en kundehenvendelse og kan være knyttet til en booking og en bolig.
- Administratorer er brugere med en særlig rolle, ikke en separat brugerentitet.

---

4. Bookingregler

4.1 Datoer og pris

Kunden vælger en ankomstdato og en afrejsedato.

- Ankomst og afrejse må ikke være samme dato.
- Afrejsedato skal ligge efter ankomstdato.
- Kunden kan tidligst ankomme den næste kalenderdag.
- Antallet af betalte døgn beregnes som forskellen mellem afrejse- og ankomstdato.
- Samlet pris beregnes som antal betalte døgn gange den aftalte dagspris.

Eksempel

Ankomst: 13. november
Afrejse: 15. november
Dagspris: 800 DKK

Betalte døgn: 2
Samlet pris: 1.600 DKK

4.2 Boligens optagede periode

Selvom afrejsedagen ikke er betalt, er boligen stadig optaget denne dag på grund af checkout og rengøring.

- Checkout senest kl. 12.00.
- Nøglen afleveres senest kl. 13.00.
- Rengøring kan begynde kl. 16.00.
- Næste kunde kan tidligst ankomme dagen efter afrejsedatoen.

I eksemplet ovenfor er boligen optaget den 13., 14. og 15. november. Næste booking kan tidligst starte den 16. november.

Brugergrænsefladen kan forklare opholdets pris som eksempelvis to betalte døgn eller 48 timers ophold. Det er alene en præsentation af varigheden; systemet bruger fortsat kalenderdatoer.

4.3 Ufravigelig regel om dobbeltbookinger

En bolig må aldrig have to bookinger, der optager samme kalenderdag.

Ved overlapkontrol skal både ankomst- og afrejsedagen betragtes som optagede. To bookinger med samme afrejsedato og ankomstdato overlapper derfor på denne dag.

En ny booking kan kun oprettes, hvis boligen er bookbar, og den ønskede periode ikke overlapper en anden aktiv reservation eller booking.

4.4 Atomisk reservation

Kontrollen af ledighed og oprettelsen af reservationen skal være beskyttet mod samtidige forespørgsler.

Den foreløbige tekniske løsning er:

1. Start en SQL Server-transaktion.
2. Tag en eksklusiv applikationslås pr. bolig, eksempelvis via `sp_getapplock`.
3. Kontrollér boligens bookbarhed, datoer og eksisterende bookinger.
4. Opret reservationen, hvis perioden er ledig.
5. Gennemfør transaktionen og frigiv låsen.

Alle kodeveje, der opretter eller ændrer bookinger, skal respektere den samme beskyttelse.

Det konkrete EF Core-design og håndtering af transaktionsfejl fastlægges under implementeringen. Integrations- og samtidighedstests skal dokumentere, at to overlappende reservationer ikke begge kan lykkes.

4.5 Reservationens udløb

- En reservation afventer betaling i højst 10 minutter.
- `ExpiresAt` gemmes som et konkret tidspunkt.
- Bookingforsøg kontrollerer udløbstidspunktet direkte.
- Et baggrundsjob markerer udløbne reservationer som `Expired`.
- En udløbet reservation må ikke genaktiveres af en sen betaling.
- En sen betaling registreres og udløser automatisk et forsøg på refundering.
- Refunderingens resultat skal registreres.

Baggrundsjobbet er oprydning og opfølgning. Det er ikke den eneste mekanisme, der sikrer korrekt ledighed.

---

5. Bookingstatusser

Status	Betydning
`PendingPayment`	Reservation oprettet, betaling afventes
`Confirmed`	Booking bekræftet efter godkendt betaling
`Cancelled`	Booking annulleret
`Completed`	Ophold afsluttet
`Expired`	Reservation udløbet uden rettidig bekræftelse

Bookingstatus er uafhængig af betalings- og refunderingstatus.

En udløbet reservation kan eksempelvis have en vellykket betaling, men må stadig ikke bekræftes.

5.1 Annullering

- Annullering mere end 24 timer før ankomst giver fuld refundering.
- Annullering 24 timer eller mindre før ankomst medfører, at kunden stadig skylder hele beløbet.
- Efter opholdets start gives der ikke refundering ved kundens annullering.
- Administratorannullering giver fuld refundering.

En annullering registrerer tidspunkt, bruger og eventuel årsag. Refunderingen skal behandles særskilt, og dens status må ikke antages at være gennemført.

Den præcise implementering af grænsetidspunktet og den efterfølgende betalings- og refunderingstilstand skal dækkes af tests.

---

6. Betalings- og refunderingregler

Betaling

- En booking kan have flere betalingsforsøg.
- Hvert forsøg gemmes separat.
- Betalingsstatus opdateres uafhængigt af bookingstatus.
- En booking bekræftes kun efter et verificeret, gyldigt betalingsresultat, mens reservationen stadig er gyldig.
- Gentagne betalingsnotifikationer må ikke skabe inkonsistente tilstande.
- En sen betaling må ikke genaktivere en udløbet booking.

Dobbeltbetalinger

Systemet skal kunne opdage, hvis der samlet er modtaget mere end bookingens beløb.

Overskydende betaling registreres og udløser et refunderingforsøg. Systemet skal undgå at oprette flere refunderinger for samme overskydende beløb ved gentagne notifikationer.

Refundering

- Refunderinger er separate poster knyttet til en betaling.
- Hver refundering har sit eget beløb og sin egen status.
- Kun bekræftede refunderinger tæller som gennemførte.
- Fejlede eller afventende refunderinger skal kunne følges op.

I V1 simuleres betalinger og refunderinger. Den virkelige betalingsudbyder og dens webhook-sikkerhed implementeres først, hvis projektet senere får behov for rigtige betalinger.

---

7. Kundehenvendelser og fejlmeldinger

Kundehenvendelser

En kunde kan sende generelle spørgsmål og bookingrelaterede henvendelser.

- Henvendelser gemmes permanent som udgangspunkt.
- `IsRead` angiver, om administratoren har læst henvendelsen.
- Dashboardet kan tælle ulæste henvendelser.
- En henvendelse kan eksistere uden en bookingrelation.
- Backend kontrollerer bookingens ejerskab, hvis en booking knyttes til henvendelsen.

Kunder kan indberette problemer i forbindelse med et ophold, mens opholdet er aktivt og op til én måned efter afslutning. Ved annullering efter opholdets start gælder fristen i op til 30 dage efter annulleringen. En booking, der annulleres før opholdets start, giver ikke adgang til denne fejlindberetning.

Officielle fejlmeldinger

- Oprettes af administratoren.
- Kan referere til en kundehenvendelse, booking og bolig.
- Har status `Open`, `InProgress` eller `Resolved`.
- Bevarer historikken, også når en fejlmelding er løst.

En kundehenvendelse er ikke automatisk en officiel fejlmelding. Administratoren vurderer først, om der er tale om et problem, som skal registreres og håndteres.

---

8. Boliger og administration

Administrator kan:

- Oprette og redigere afdelinger og boliger.
- Tilføje og sortere boligbilleder.

- Ændre beskrivelse, kapacitet, egenskaber og dagspris.
- Publicere og afpublicere boliger.
- Slå `IsBookable` til eller fra.
- Se og administrere bookinger.
- Behandle kundehenvendelser og oprette fejlmeldinger.

En ændring af boligens dagspris påvirker kun nye bookinger. Eksisterende bookinger bevarer deres aftalte dagspris og samlede beløb.

V1 gemmer ikke en generel ændringshistorik for boligoplysninger.

Når en bolig gøres ikke-bookbar, må nye bookinger afvises. Eksisterende bookinger annulleres ikke automatisk. Administratoren skal tage stilling til dem særskilt.

---

9. Sikkerhed og dataintegritet

Følgende regler er obligatoriske:

- Backend håndhæver adgangskontrol og validering.
- Kunder kan kun se og ændre egne beskyttede data.
- Administratorfunktioner kræver den relevante rolle.
- Betalingskortoplysninger gemmes ikke.
- Hemmeligheder og adgangskoder må ikke hardcodes i kildekoden.
- Bookingens pris gemmes som et øjebliksbillede.
- Overlappende bookinger må ikke kunne bekræftes eller reserveres samtidigt.
- En udløbet reservation må ikke genaktiveres af en sen betaling.
- Betalings- og refunderingstatusser må ikke forveksles med bookingstatus.
- Gentagne betalingsnotifikationer må ikke føre til utilsigtede dobbeltbetalinger eller dobbeltrefunderinger.
- Historikken over betalingsforsøg, refunderinger og officielle fejlmeldinger skal bevares.

---

10. Oprettelsestidspunkter

De centrale forretningsentiteter gemmer et oprettelsestidspunkt:

- `Booking`
- `Payment`
- `Refund`
- `Inquiry`

- `FaultReport`

Tidspunkter repræsenterer konkrete hændelser og gemmes som tidsstempler, der kan håndteres korrekt på tværs af tidszoner. I C# vil `DateTimeOffset` være et naturligt valg.

Vi kræver ikke automatisk `CreatedAt` på alle hjælpeentiteter.

---

11. Åbne implementeringsbeslutninger

Følgende fastlægges, når vi bygger de relevante funktioner:

1. De præcise C#-properties og databasetyper.
2. De endelige værdier og overgangsregler for betalings- og refunderingstatusser.
3. Det konkrete EF Core-design for transaktioner og applikationslåse.
4. Om vi bruger controllers eller Minimal APIs.
5. De konkrete API-endpoints, DTO'er og fejlformater.
6. Den konkrete lokale fillagring og senere cloud-lagring.
7. Det tekniske baggrundsjob til udløbne reservationer.
8. Den endelige håndtering af e-mails og notifikationsfejl.
9. De nødvendige databaseindekser og øvrige performanceoptimeringer.

Disse beslutninger skal ikke forhindre os i at begynde implementeringen. De træffes i forbindelse med den relevante funktion og dokumenteres, når de er afgjort.

---

12. Klar til implementering

Domænemodellen giver nu et fælles grundlag for at bygge systemet. Den første implementeringsfase bør være en lille vertikal slice, som forbinder frontend, API og database.

Foreslået rækkefølge:

1. Opret solution og projekter.
2. Opsæt SQL Server lokalt via Docker Compose.
3. Opsæt ASP.NET Core Web API og Angular.
4. Tilføj EF Core og opret den første migration.
5. Implementer en lille funktion fra Angular gennem API'et til databasen.
6. Tilføj tests og dokumentation, efterhånden som funktionerne bygges.

Vi udvider derefter gradvist med brugeridentitet, boliger og til sidst det mere komplekse bookingforløb.

Princip: Vi dokumenterer forretningsreglerne på forhånd, men implementerer og verificerer dem i små, testbare trin.